[TODO(check): verify CCS concept ids at dl.acm.org/ccs]

Gating Publish/Subscribe Misconfigurations in CI/CD with an Architectural Graph Model: Experience from a Naval Combat Management System

Ibrahim Onuralp Yigit
Command Control and Defense Technologies, HAVELSAN
Istanbul, Turkiye
iyigit@havelsan.com.tr

Onurcan Ersen
Command Control and Defense Technologies, HAVELSAN
Istanbul, Turkiye
oersen@havelsan.com.tr

Mustafa Can Caliskan
Command Control and Defense Technologies, HAVELSAN
Istanbul, Turkiye
mccaliskan@havelsan.com.tr

Emre Karagoz
Software Development Center, TNRCC
Istanbul, Turkiye
karagoz.e5926@dzkk.tsk.tr

Omer Kursat Ucarer
Software Development Center, TNRCC
Istanbul, Turkiye
ucarer.o1595@dzkk.tsk.tr

Abstract

A naval combat management system (CMS) is hundreds of applications exchanging data over DDS-style publish/subscribe middleware. Its costliest integration defects are architectural misconfigurations that no unit test sees: a data writer that offers weaker Quality-of-Service (QoS) than its reader requests and is therefore never matched, a topic left without subscribers after a refactor, a dependency cycle that deadlocks start-up, a host whose cores are over-committed. Today these surface at shipyard integration or sea trials.

We report on SaaG, a CI/CD gate built at HAVELSAN that rebuilds each candidate release as a typed dependency graph from its configuration descriptors, checks it against middleware rules, and blocks builds that introduce new severe violations. We evaluate it four ways: injecting 12 classes of faulty and 7 classes of benign mutations into five generated platform profiles, with QoS verdicts cross-checked against the middleware's own endpoint matching; running it on a fielded baseline whose findings two architects triaged; replaying ? historical integration defects; and timing it. The gate detects ? of the injected faults within its implemented rules at a false-positive rate of ?, flags ? of ? historical defects, and adds 16–100 s to a build. The binding constraint was not analysis but data: one baseline drew on ? configuration sources in ? formats and took ? person-weeks to curate. We report what that means for organizations adopting such gates.

CCS Concepts

• **Software and its engineering → Software verification and validation**; *Distributed systems organizing principles.*

Keywords

publish/subscribe middleware, DDS QoS, architecture conformance, configuration errors, CI/CD gating, fault injection, defense systems

1 Introduction

Naval combat management systems (CMS) used to be monolithic mainframe software. Today they are open, distributed architectures: radar trackers, track fusion, threat evaluation, weapon assignment, fire control and operator consoles run as separate applications on mission computers and consoles, and exchange data over publish/subscribe middleware that follows the OMG Data Distribution Service (DDS) model [8]. The CMS we work on, developed by HAVELSAN with the Turkish Naval Forces Research Center Command, serves surface combatants, amphibious ships, unmanned vehicles and maritime patrol aircraft, with hundreds of applications per platform.

Continuous integration [5] checks each of these applications well and their composition poorly. Unit and component tests run one application at a time, so a class of defects passes every test and only appears when the whole system is assembled:

- *QoS mismatch.* In DDS, a reader states the QoS it *requests* and a writer the QoS it *offers*; if the offer is weaker (e.g., the weapon assignment writer offers `VOLATILE` durability while fire control requests `TRANSIENT_LOCAL`), the middleware never connects them. No exception is raised; data simply never arrives.
- *Orphaned topic.* A refactor removes the last subscriber of a topic that another subsystem still publishes, or vice versa.
- *Dependency cycle.* Two subsystems each wait for the other's data at start-up and deadlock the console.
- *Core over-commitment.* Real-time applications pinned to a host request more cores than the host has.

Finding these needs the assembled target: consoles, mission computers, sensor and weapon simulators. That hardware cannot sit in a CI pipeline, so the defects are found at shipyard integration, harbor acceptance tests (HAT) or sea acceptance tests (SAT), where each costs days of a scarce test window.

SaaG (System as a Graph) moves these checks into the pipeline. On each merge request it rebuilds the candidate release as a typed graph from configuration descriptors—which application runs where, publishes and subscribes to what, with which QoS—derives who depends on whom, checks rules over that graph, and fails the build when the candidate adds a severe violation. It reuses the graph model of Yigit and Buzluca [20]; what is new here is the gate built on it and the evidence about it.

Showing that such a gate is cheap does not show that it is useful. This paper therefore asks:

RQ1 Does the gate detect injected misconfigurations, and does it accept valid changes?
RQ2 What does it report on a real fielded baseline, and are those findings real?
RQ3 Would it have caught defects that historically escaped to integration and sea trials?
RQ4 What does it cost in the pipeline?

Contributions.

(1) A CI/CD gate for DDS-style pub/sub systems (§3): a rule catalog over request/offered QoS, partitions, topic continuity, dependency cycles and core capacity; a severity rubric adapted from MIL-STD-882E [2]; and gate semantics for a legacy codebase (fail-closed policy, delta-aware gating, waiver register). Table 2 separates this from what [20] provides.
(2) An effectiveness evaluation (§4–§5): fault injection with an independent middleware oracle, a triaged fielded baseline, historical defect replay, cost, and a sensitivity analysis of the severity rubric.
(3) Evidence on the real adoption barrier (§6): configuration data is spread across organizational and security silos, and acquiring it—not analyzing it—dominated the effort. We quantify that effort and give recommendations.

Deployment status. SaaG runs as a CLI gate on a GitLab CI runner over hand-curated JSON descriptors exported from application repositories. It is not yet fed from an authoritative configuration source; §6 explains why. We distinguish throughout between rules the prototype implements and capabilities that are specified but not built.

2 Background and Running Example

2.1 DDS QoS matching in one paragraph

DDS applications exchange typed samples on named *topics*. Each data writer and data reader carries QoS policies. For a subset of policies, the specification defines a *request/offered* (RxO) rule: a writer and a reader are matched only if what the writer offers is at least what the reader requests [8]. For `DURABILITY` the order is `VOLATILE` < `TRANSIENT_LOCAL` < `TRANSIENT` < `PERSISTENT`; for `RELIABILITY`, `BEST_EFFORT` < `RELIABLE`; `DEADLINE`, `LATENCY_BUDGET`, `LIVELINESS`, `OWNERSHIP` and `DESTINATION_ORDER` are RxO as well. `PARTITION` is not ordered: writer and reader must share a partition name. An unmatched pair is reported only through an incompatible-QoS status that applications must poll or listen for—in practice, rarely—so the failure is silent. Middleware tools show these statuses at run time, which for a CMS means on a testbed or on the ship.

2.2 The industrial setting

The CMS middleware is proprietary but follows the DDS data-centric model and RxO semantics, so we use DDS terms throughout. A platform baseline is the set of application versions, their hosts and their middleware configuration for one ship class. Configuration lives in many places: application repositories, middleware QoS profiles, start-up scripts, interface control documents (ICDs) for sensors and weapons, and shipyard commissioning archives (§6). Five platform classes, from a single-node maritime patrol aircraft to a corvette with distributed consoles, define our evaluation subjects (§4.1).

2.3 Running example

Figure 1 follows one change through the gate. Three applications run on two hosts: *Track Fusion* (TF) publishes `track`; *Weapon Assignment* (WA) subscribes to `track` and publishes `engagement_order`; *Fire Control* (FC) subscribes to `engagement_order` and requests `TRANSIENT_LOCAL` durability so that it receives the current order even if it restarts. A merge request changes WA's writer to `VOLATILE`. Every unit test of WA still passes.

SaaG reads the candidate's descriptors (a), builds the structural graph (b), and projects dependency edges in the direction failures propagate—FC depends on WA because FC starves if WA stops, not the reverse (c). The RxO rule compares FC's request with WA's offer on `engagement_order`, finds the offer weaker, and, because the channel is tagged safety-critical, rates the violation S1. The violation is absent from the baseline, so the gate exits 1 and the merge request is blocked with the message in (d).

3 SaaG: Model, Rules and Gate

3.1 Graph model (from [20])

SaaG represents a platform baseline as a typed directed multigraph $G = (V, E, \tau_V, \tau_E)$. Vertices are *applications*, *brokers* (middleware routing services), *topics*, *hosts* (mission computers, consoles, server chassis) and shared *libraries*. Structural edges are read directly from descriptors: `PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO` and `USES`.

Message flow runs from publisher to subscriber, but failure dependency runs the other way: if a publisher stops, its subscribers starve; if a subscriber stops, the publisher is unaffected. Six projection rules (Table 1) derive `DEPENDS_ON` edges from the structural edges. The cycle rule (§3.2) runs on these projected edges, and diagnostic mode (§3.5) uses them to report what a change can affect.

Each vertex also carries a criticality weight $w(v) \in [0, 1]$, computed in [20] from the QoS of the topics it touches, message size and fan-out. In the gate the weights do one thing: they order findings *within* a severity level for triage. They do not decide pass or fail; §5.6 tests how sensitive triage order and severities are to them. **[TODO(check): confirm against protocol decision D2 that weights only order triage; if they feed severity, rewrite this paragraph]**

[TODO(check): confirm the split in Table 2 with the RASSE'25 paper; adjust if any rule was introduced there]

3.2 Rules, as implemented

Table 3 lists the rules the prototype runs, stated at the granularity at which they are actually implemented. The narrowings matter, and RQ1 measures their cost: the RxO rule compares QoS per *topic*, not per writer–reader pair **[TODO(check): state exactly how a topic's offer and request are aggregated]**; the schema rule matches topic names, not message fields; the core rule checks that requested cores fit host capacity, not that core masks do not overlap.

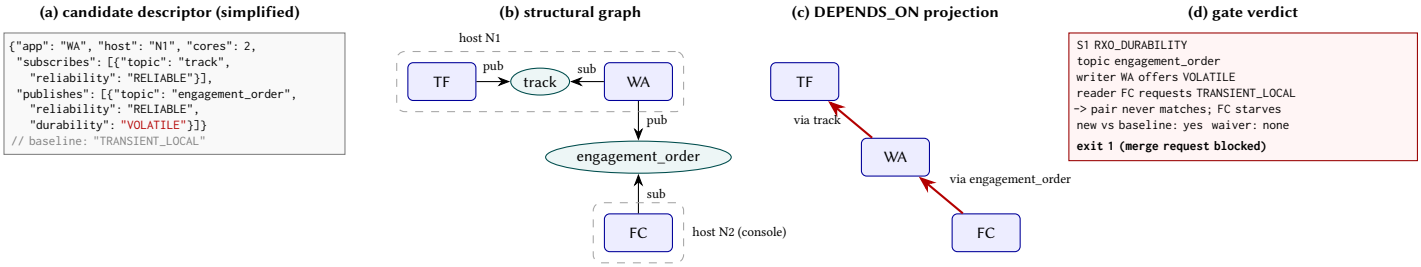


Figure 1: Running example. A one-field change in a writer's QoS (a) passes unit tests. SaaG builds the structural graph (b), projects failure dependencies opposite to data flow (c), finds the request/offered durability violation on a safety-critical channel, and blocks the merge request (d).

Table 1: `DEPENDS_ON` projection rules (from [20]). $X \rightarrow Y$ means X depends on Y.

#	Type	Derived when
1	app → app	B subscribes to a topic that A publishes ($B \rightarrow A$)
2	app → broker	A publishes or subscribes to a topic that broker R routes
3	host → host	Rule 1 lifted to the hosts of B and A
4	host → broker	Rule 2 lifted to the host of A
5	app → library	A links library ℓ (`USES`)
6	broker → broker	Two brokers share a host

Table 2: What this paper adds to the graph model of [20].

Element	[20]	This paper
Entity/edge types, `DEPENDS_ON` projection	•	reused
QoS-derived criticality weights	•	reused for triage order
Candidate isolation $G_{u'}$ per merge request		•
Middleware rule catalog (Table 3)		•
Severity rubric adapted from MIL-STD-882E		•
Gate semantics: exit codes, fail-closed, delta-aware, waivers		•
Effectiveness evaluation (RQ1–RQ3)		•
Data-acquisition study		•

3.3 Severity

MIL-STD-882E [2] grades the severity of mishaps, not of configuration defects, so we adapt its four levels: a violation is rated by the worst credible mishap it could contribute to, given which channels and hosts it touches. Channels and hosts carry a safety-criticality tag maintained by the system safety team. **[TODO(check): D2: state the exact mapping]**

- **S1** — loss of a safety-critical function (e.g., unmatched pair on a weapon engagement channel; over-committed fire control host).
- **S2** — loss of a mission-essential function with redundancy or mitigation (e.g., orphaned data-link forwarding topic; cycle among engagement planning units).
- **S3** — non-critical anomaly (e.g., orphaned auxiliary sensor stream).

Table 3: Rules implemented in the prototype (SaaG-P).

Rule	Check, as implemented	Typical sev.
RxO	Per topic: offered ≥ requested for `DURABILITY`, `RELIABILITY`; shared `PARTITION` **[TODO(check): D1: + DEADLINE, LIVELINESS, …?]**	S1–S2
Continuity	Topic with no publisher or no subscriber	S2–S3
Schema	Publisher and subscriber agree on the topic name	S2
Cycle	Strongly connected component (Tarjan) in `DEPENDS_ON` among applications	S2
Capacity	$\sum$ requested cores of units on a host ≤ host cores	S1–S3

- **S4** — maintenance warning (e.g., deprecated library version).

Severity is deliberately independent of the middleware's transport priority: a low-priority channel can still feed a safety-critical function.

3.4 Gate semantics

For each merge request, SaaG substitutes the candidate version u' of the changed unit into the baseline inventory, giving an isolated candidate graph $G_{u'}$; the baseline is never modified. The gate exits 0 (pass; S3/S4 logged), 1 (an S1 or S2 violation; pipeline aborted) or 2 (the descriptors could not be parsed or analyzed). On safety-critical branches the gate is *fail-closed*: exit 2 also aborts, so an unanalyzable candidate never passes unverified.

A legacy baseline already carries violations, and an absolute gate would block every build. SaaG therefore gates on the *delta*: a candidate is blocked only if $\mathrm{Viol}(G_{u'}) \setminus \mathrm{Viol}(G_{\mathrm{base}})$ contains an S1/S2 violation. Known baseline violations live in a waiver register; each waiver is approved by the chief architect, signed, and expires.

3.5 Diagnostic mode

The same graph serves architects outside the pipeline: SCCs show cycles to break; orphaned topics and unreferenced libraries show configuration debt; high-weight brokers and topics show where a change has wide reach; and a proposed change can be applied to a copy of the baseline graph to see what it affects. §5.3 reports what this surfaced on a fielded baseline and what the team did with it.

4 Study Design

4.1 Subjects

Generated profiles. Operational configuration data of fielded combat systems is not releasable, and §6 explains why it was also hard to obtain internally. We therefore generate five platform profiles whose scale parameters—application, topic, host and library counts, and the distributions of QoS settings and fan-out—were elicited from the architects of the corresponding platforms: Maritime Surveillance Aircraft (1 host), Coastal Surveillance (25), Unmanned Surface Vehicle (1), Amphibious Ship (65) and Corvette (20). They span 180–320 applications, 1,200–2,800 topics and 1,500–3,300 vertices. Each profile is generated with 5 seeds, giving 25 baselines. Generated baselines are violation-free by construction; we confirm this with the gate (§5.2).

Platform R. One fielded platform baseline, anonymized as *Platform R* under the clearance recorded in our data package: **[TODO(data): clearance.md – anonymization sentence]**. Its descriptors were curated by hand from the sources described in §6.

Historical defects. **?** architectural defects drawn from the program's defect tracker by a selection rule fixed before running the gate: **[TODO(data): rq3_replay.csv – selection_rule]**.

4.2 RQ1: fault injection

Each mutant applies one mutation to one clean baseline (Table 4). Faulty operators M1–M12 cover the fault classes the rules target, plus three that target the known narrowings of Table 3 (M10–M12), so the cost of each narrowing is measured rather than asserted. Benign operators B1–B7 make changes a correct gate must accept. Mutation sites are drawn uniformly, stratified so that at least a third lie on safety-critical channels. We generate 40 mutants per operator per profile (200 per operator).

A finding counts as a detection only if it is *at* the mutated element; findings elsewhere count as false positives. We report per-operator recall and false-positive rate with Wilson 95% intervals, gate-decision accuracy (exit 1 exactly when the expected severity is S1/S2), and severity agreement.

Oracle. For the QoS operators, our expected outcome is our own reading of the specification. To check that reading independently, we instantiate 50 sampled writer–reader pairs per operator in **[TODO(data): rq1_oracle.csv – middleware]** and record whether the middleware reports incompatible QoS and whether the pair matches.

4.3 RQ2: fielded baseline

We run the gate on Platform R without baseline subtraction. Two architects triage each finding independently (on a sample of at least 30 findings) as *TP-fix* (real; fixed or scheduled), *TP-debt* (real; accepted into the waiver register), *FP-model* (the rule is wrong for this case) or *FP-data* (a descriptor was wrong or stale); we report Cohen's κ and resolve disagreements jointly. We then replay the last **?** merged change sets with absolute and with delta-aware gating and count blocks under each.

Table 4: Mutation operators. Exp.: expected outcome on SaaG-P (+ detect; × known miss; ✓ accept).

Op	Mutation	Exp.
M1	Writer durability below a reader's request	+
M2	`BEST_EFFORT` writer, `RELIABLE` reader	+
M3	Redundant publisher moved to another `PARTITION`	+
M4	Offered `DEADLINE` period longer than requested	**[TODO(check): D1]**
M5	Offered `LIVELINESS` weaker than requested	**[TODO(check): D1]**
M6 / M7	Last subscriber / last publisher of a topic removed	+
M8	Dependency edge closing a cycle (length 2, 3, ≥5)	+
M9	Requested cores on a host exceed capacity	+
M10	Overlapping core masks within capacity	×
M11	Same topic name, different message fields	×
M12	One of several writers on a topic weakened	+/?
B1 / B2	Writer QoS raised / reader QoS lowered	✓
B3	Compatible subscriber added	✓
B4 / B5	Consistent rename / reordering of descriptors	✓
B6	Unit moved to a host with spare cores	✓
B7	Edge added that closes no cycle	✓

4.4 RQ3: historical replay

For each selected defect, we reconstruct the pre-fix descriptors, run the gate, and record whether it flags the defect, with which rule and severity, and—if not—why: capability not implemented, information absent from descriptors, or outside the gate's scope.

4.5 RQ4 and sensitivity

Latency is the mean of 500 evaluations per profile across the 5 seeds after 20 warm-up runs, on Ubuntu 22.04 (AMD EPYC 7763, 32 GB), Python 3.11, NetworkX 3.2, in memory. For sensitivity, we perturb each weight parameter and QoS factor value by ±10% and ±20% one at a time, plus 200 random joint perturbations within ±20%, and measure on the RQ1 mutants and on Platform R how many severities change, how many gate decisions flip, and Kendall's τ between triage orders.

5 Results

5.1 Overview

[TODO(data): two-sentence summary of RQ1–RQ4 once all files are filled]

5.2 RQ1: detection and acceptance

[TODO(data): rq1_mutants.csv – clean baselines: number of findings on the 25 unmutated baselines and their cause]

Table 5: RQ1 results, pooled over 5 profiles × 5 seeds (n = 200 per operator). Recall and FP rate with Wilson 95% CI. [TODO(data): rq1_mutants.csv]

Op	Rule	Recall	Gate acc.	Sev. agr.
M1	RxO	?	?	?
M2	RxO	?	?	?
M3	RxO (partition)	?	?	?
M4 / M5	RxO (extended)	?	?	?
M6 / M7	Continuity	?	?	?
M8	Cycle	?	?	?
M9	Capacity	?	?	?
M10	(non-overlap, not impl.)	?	?	?
M11	(field schema, not impl.)	?	?	?
M12	RxO, per topic	?	?	?
		FP rate	Gate acc.	
B1–B7	all	?	?	
clean	all	?	?	

Table 6: RQ2: absolute audit of Platform R, with architect triage. [TODO(data): rq2_baseline_findings.csv]

Rule	Findings	TP-fix	TP-debt	FP-model	FP-data
RxO	?	?	?	?	?
Continuity	?	?	?	?	?
Schema	?	?	?	?	?
Cycle	?	?	?	?	?
Capacity	?	?	?	?	?
Total	?	?	?	?	?

Table 5 gives the results per operator. **[TODO(data): rq1_mutants.csv – Platform R latency] – recall of implemented rules (M1–M3, M6–M9), pooled with CI; FP rate on B1–B7]**

[TODO(data): rq1_mutants.csv – M10–M12: measured miss rates; for M12, how often the flagged writer was the mutated one]

[TODO(data): rq1_oracle.csv – agreement with the middleware; list and explain every disagreement]

5.3 RQ2: findings on Platform R

[TODO(data): rq2_baseline_findings.csv – precision on Platform R; Cohen's κ; median triage minutes; what the FP-model findings have in common]

[TODO(data): rq2_delta_gating.csv – of ? change sets, blocked under absolute vs. delta-aware gating; how many delta blocks were justified; waiver entries created]

Diagnostic mode. [TODO(data): rq2_diagnostic_outcomes.csv – SCCs, orphaned topics, unreferenced libraries found, and what was actually refactored or ticketed. Report only outcomes that happened.]

Table 7: RQ3: replay of historical defects. Found: phase in which the defect was found. [TODO(data): rq3_replay.csv]

ID	Fault class	Found	Flagged	Rule / reason missed
D1	?	?	?	?
D2	?	?	?	?
…				

Table 8: RQ4: gate latency (s). Generated rows: mean of 500 runs over 5 seeds; counts approximate.

Profile	Hosts	$\|V\|$	$\|E\|$	Constr.	Total	P95
Maritime Surv. Aircraft	1	1500	4800	11.11	15.53	17.27
Coastal Surveillance	25	2200	7100	24.49	38.57	38.94
Unmanned Surface Veh.	1	2200	7000	23.53	45.42	47.63
Amphibious Ship	65	2800	9600	44.52	85.87	87.68
Corvette	20	3300	11500	52.34	100.16	102.31
Platform R	?	?	?	?	?	?

5.4 RQ3: historical defects

[TODO(data): rq3_replay.csv – flagged k of n; for the misses, which were implementation gaps (Table 10) vs. missing data vs. out of scope; cost bucket of flagged defects]

5.5 RQ4: cost

Table 8 gives gate latency. On the generated profiles the gate takes 15.5 s (single-host aircraft profile) to 100.2 s (corvette), P95 ≤ 102.3 s. Graph construction is 52–72% of it; rule evaluation and dependency projection the rest. Latency grows with vertex and edge count; the 65-host amphibious profile is slower than its size alone suggests because rules 3–4 lift dependencies across many hosts. Against the program's release pipeline, which takes about 30 minutes (**[TODO(data): rq4_latency.csv – full_pipeline: n and window]**), the gate adds at most 5.6%. **[TODO(data): rq4_latency.csv**

5.6 Sensitivity of severity and triage

[TODO(data): sens_weights.csv – fraction of severities changed and gate decisions flipped under ±10/20% and joint perturbation; Kendall τ of triage order. If severity does not depend on weights (D2), state that gate decisions are invariant by construction and report only τ.]

6 Industrial Experience: Data, Not Analysis, Is the Bottleneck

RQ4 shows the analysis is cheap. Building the input was not. This section reports what it took to curate Platform R's descriptors and why the prototype is still not fed automatically.

6.1 Where configuration lives

The information one baseline needs is split across three kinds of silo. *Engineering silos*: application code and some QoS settings live in Git, but QoS is also set in middleware XML profiles, in code, and in start-up scripts, with no single authoritative form. *Security*

Table 9: Data acquisition for Platform R. [TODO(data): lessons_effort.csv]

Source	Format	Lead time (d)	Conflicts
Application repositories	?	?	?
Middleware QoS profiles	?	?	?
Start-up scripts / core bindings	?	?	?
Sensor/weapon ICDs	?	?	?
Asset database	?	?	?
Total curation effort	? person-weeks		

and contractual silos: sensor and weapon ICDs and the hardware core bindings are held in shipyard commissioning archives under separate release control, some owned by subcontractors. *Inventory silos*: the enterprise asset database is built for manual audit and has no machine-readable export, so a CI runner cannot ask it what a target host looks like.

Table 9 quantifies the cost for Platform R. **[TODO(data): lessons_effort.csv – number of sources and formats; fields reconciled by hand and conflicts found; person-weeks; access lead times; approvals refused]**

Curation itself found defects. [TODO(data): lessons_effort.csv – descriptor_errors_found: inconsistencies between sources discovered while building the model, and whether any were real configuration errors]

Organizational obstacles. [TODO(data): lessons_effort.csv – notes: which approvals were needed, who owned which source, which requests were refused or delayed, and how that shaped what the prototype could check]

6.2 What the silos cost in capability

Table 10 lists the capabilities our architects specified, what the prototype realizes, what blocks the rest, and—new in this version—what that gap costs as measured in §5. Six of seven gaps are blocked by data, not by algorithms: each missing capability is a check we know how to write but cannot feed.

6.3 Recommendations

(1) **Make configuration declarative before buying analysis.** A single machine-readable source for QoS, partitions and core bindings would remove blockers 1, 3 and 7; the graph analysis already exists.
(2) **Treat ICDs as code.** Machine-readable interface definitions in a registry the CI runner can read would remove blocker 2 and make schema checks field-level.
(3) **Gate on the delta from day one. [TODO(data): rq2_delta_gating.csv – one sentence with the absolute vs. delta block counts]**
(4) **Expect curation to find defects. [TODO(data): lessons_effort.csv – one sentence]**

7 Threats to Validity

Construct. Mutation operators encode our view of the fault classes; RQ3 checks that view against defects that actually occurred. Expected outcomes for QoS operators come from our reading of the DDS specification, which the middleware oracle checks independently (§4.2).

Internal. Generated topologies may be more regular than real ones; Platform R is the check against that. Triage on Platform R involves judgment; two independent architects and κ bound it. Descriptor curation is manual and may itself introduce errors; findings caused by such errors are classified FP-data rather than hidden.

External. One organization, one middleware, one product line. The rules depend only on DDS RxO semantics and on descriptor content, so they transfer to other DDS and ROS 2 systems that can export the same descriptors; the data-acquisition findings are specific to defense programs with security-separated configuration, though we expect them in other regulated domains.

Conclusion. Wilson intervals are reported for all rates; with 200 mutants per operator, the half-width is at most about 7 percentage points.

8 Related Work

Architecture recovery and conformance. Reflexion models compare an intended architecture with one extracted from code [7]; erosion and its control are well studied [1, 10], as are dependency constraint languages [15] and test-style conformance tools such as ArchUnit [17]. These target code-level structure; SaaG checks the run-time communication architecture as configured.

Pub/sub and robotics misconfiguration. Closest to us, ROS-Discover [16] statically recovers the run-time architecture of ROS systems from code and launch files and checks it against rules in first-order logic, and HAROS [11] extracts models of ROS applications for further analysis. Both recover the architecture from source; SaaG assumes descriptors and instead contributes the CI gate semantics, the DDS RxO and partition rules, safety-graded severity, and the evidence from a defense setting where—as §6 shows—getting the descriptors is the hard part. DDS and ROS 2 tools report incompatible QoS at run time [8], which for a combat system means on a testbed or ship.

Configuration errors. Configuration errors are a major cause of failures [19]. PCheck generates checkers that catch latent errors at start-up [18]; ctest tests configuration changes against the code that uses them [12]; Facebook [13] and ConfValley [4] validate configuration as code before deployment; policy engines such as OPA Gatekeeper [9] gate cluster manifests. SaaG applies the same shift-left idea to cross-application middleware contracts, which none of these models.

Model-based analysis of embedded architectures. AADL and its tool set analyze resource budgets, bindings and flows of embedded system models [3]. Such models are rarely maintained for a CMS whose configuration evolves per merge request; SaaG derives its model from the release itself.

Digital models. In the taxonomy of Kritzinger et al. [6] and the review of Tao et al. [14], SaaG is a digital *model*: built from static artifacts, with no data flowing back from the running system. A runtime drift check (Table 10, row 5) would make it a digital shadow.

9 Conclusion

SaaG gates a naval CMS's merge requests on publish/subscribe architecture rules: request/offered QoS, partitions, topic continuity,

Table 10: Specified vs. realized capabilities, their blockers, and the measured consequence.

Specified capability	Realized in SaaG-P	Blocker	Measured consequence
1. Endpoint-level RxO matching	Per-topic	**Data:** endpoint QoS held in vendor tool chains	M12 (§5.2): ?
2. Field-level schema alignment	Topic-name match	**Data:** sensor/weapon ICD parsers not on the CI runner	M11: ?
3. Core mask non-overlap	Host capacity	**Data:** core bindings in scripts outside application repos	M10: ?; RQ3: ?
4. OS memory/kernel parameter audit	Not built	**Data:** no programmatic host profiles	RQ3: ?
5. Runtime drift (observed vs. designed edges)	Not built	**Data:** no field telemetry repository	not evaluated
6. Scored release suitability	Exit-code gate	**Calibration:** penalty weights need cross-organization agreement	§5.6
7. Automated topology ingestion	Hand-curated JSON	**Data:** asset database has no machine-readable export	Table 9

dependency cycles and core capacity. Injected faults show that **[TODO(data): one-sentence RQ1 headline]**; on a fielded baseline **[TODO(data): RQ2 headline]**; and replaying historical defects **[TODO(data): RQ3 headline]**. The gate costs at most 100 s per build. The binding constraint is configuration data: it is scattered across engineering, contractual and security silos, and six of seven capabilities our architects asked for are blocked by it. Our next steps are therefore ingestion rather than analysis: harvesting ICDs and core bindings automatically, and—once field telemetry exists—the runtime feed that would turn this digital model into a digital shadow.

Data Availability

The prototype and the fielded configuration data are proprietary and covered by defense release restrictions. **[TODO(data): clearance.md – list what is released (e.g., generator parameters, mutation catalog, synthetic profiles and mutants, analysis scripts) and where; state what is withheld and why.]**

Acknowledgments

Generative AI tools were used in this work solely for grammar and language polishing. The authors retain full responsibility for all technical content, claims, data, and results presented in this paper.

References

[1] L. de Silva and D. Balasubramaniam. 2012. Controlling Software Architecture Erosion. *Journal of Systems and Software* 85, 1 (2012), 132–151.
[2] Department of Defense. 2012. *System Safety (MIL-STD-882E).* DoD Standard Practice. U.S. Department of Defense.
[3] Peter H. Feiler and David P. Gluch. 2012. *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language.* Addison-Wesley.
[4] P. Huang et al. 2015. ConfValley: Validating System Configurations. In *Proceedings of ACM EuroSys.* Article 4.
[5] J. Humble and D. Farley. 2010. *Continuous Delivery.* Addison-Wesley.
[6] W. Kritzinger et al. 2018. Digital Twin in Manufacturing: A Categorical Literature Review. *IFAC-PapersOnLine* 51, 11 (2018), 1016–1022.
[7] G. C. Murphy, D. Notkin, and K. J. Sullivan. 2001. Software Reflexion Models. *IEEE Transactions on Software Engineering* 27, 4 (2001), 364–380.
[8] Object Management Group (OMG). 2015. Data Distribution Service (DDS) Specification. Version 1.4.
[9] Open Policy Agent. [n. d.]. OPA Gatekeeper: Policy Controller for Kubernetes. https://open-policy-agent.github.io/gatekeeper/. Accessed 2026-11-03.
[10] D. E. Perry and A. L. Wolf. 1992. Foundations for the Study of Software Architecture. *ACM SIGSOFT Software Engineering Notes* 17, 4 (1992), 40–52.
[11] André Santos, Alcino Cunha, and Nuno Macedo. 2021. The High-Assurance ROS Framework. In *Proceedings of the 3rd IEEE/ACM International Workshop on Robotics Software Engineering (RoSE).* 37–40. doi:10.1109/RoSE52553.2021.00013
[12] Xudong Sun, Runxiang Cheng, Jianyan Chen, Elaine Ang, Owolabi Legunsen, and Tianyin Xu. 2020. Testing Configuration Changes in Context to Prevent Production Failures. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI).* 735–751.
[13] C. Tang et al. 2015. Holistic Configuration Management at Facebook. In *Proceedings of ACM SOSP.* 328–343.
[14] F. Tao et al. 2019. Digital Twin in Industry: State-of-the-Art. *IEEE Transactions on Industrial Informatics* 15, 4 (2019), 2405–2415.
[15] R. Terra and M. T. Valente. 2014. A Dependency Constraint Language. *Software: Practice and Experience* 44, 9 (2014), 1073–1094.
[16] Christopher S. Timperley, Tobias Dürschmid, Bradley Schmerl, David Garlan, and Claire Le Goues. 2022. ROSDiscover: Statically Detecting Run-Time Architecture Misconfigurations in Robotics Systems. In *Proceedings of the 19th IEEE International Conference on Software Architecture (ICSA).* 112–123. doi:10.1109/ICSA53651.2022.00019
[17] TNG Technology Consulting. [n. d.]. ArchUnit: Unit Test Your Java Architecture. https://www.archunit.org. Accessed 2026-11-03.
[18] Tianyin Xu, Xinxin Jin, Peng Huang, Yuanyuan Zhou, Shan Lu, Long Jin, and Shankar Pasupathy. 2016. Early Detection of Configuration Errors to Reduce Failure Damage. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI).* 619–634.
[19] T. Xu and Y. Zhou. 2015. Systems Approaches to Tackling Configuration Errors. *Comput. Surveys* 47, 4 (2015), 70.
[20] I. O. Yigit and F. Buzluca. 2025. A Graph-Based Dependency Analysis Method for Identifying Critical Components in Distributed Publish-Subscribe Systems. In *IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE).* doi:10.1109/RASSE64831.2025.11315354